ie UNIVERSITY

INTRODUCTION TO ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

MCSBT

Master in Computer Science & Business Technology

john E. Ortega, PhD | Fall 2026 Semester 1 – Week 3 Session 7



Class Plan

Session #	Date	Topic
1	**Sep 28**	AI Foundations and Technology Landscape
2	**Sep 30**	AI Strategy and Business Transformation
3	**Oct 2**	Machine Learning Fundamentals (assignment 1)
4	**Oct 5**	Deep Learning
5	**Oct 7**	NLP and Foundation Models (assignment 2)
6	**Oct 14**	Large Language Models (Quiz 1)
7	**Oct 16**	Retrieval-Augmented Generation (RAG)
8	**Oct 19**	Agentic AI Systems (assignment 3)
9	**Oct 21**	Multimodal AI
10	**Oct 26**	Recommender Systems (Quiz 2)
11	**Oct 28**	Cloud AI, MLOps and LLMOps (assignment 4)
12	**Oct 30**	Responsible AI and Governance
13	**Nov 4**	Emerging Topics (assignment 5)
14-15	**Nov 11 + 13**	Final Presentations

Session 7 Agenda

- Introduction and Embeddings
- Vector Databases
- Semantic Search
- Knowledge Graphs
- Retrieval Strategies
- RAG Evaluation and Hallucination
- Enterprise Search
- Build a RAG Pipeline

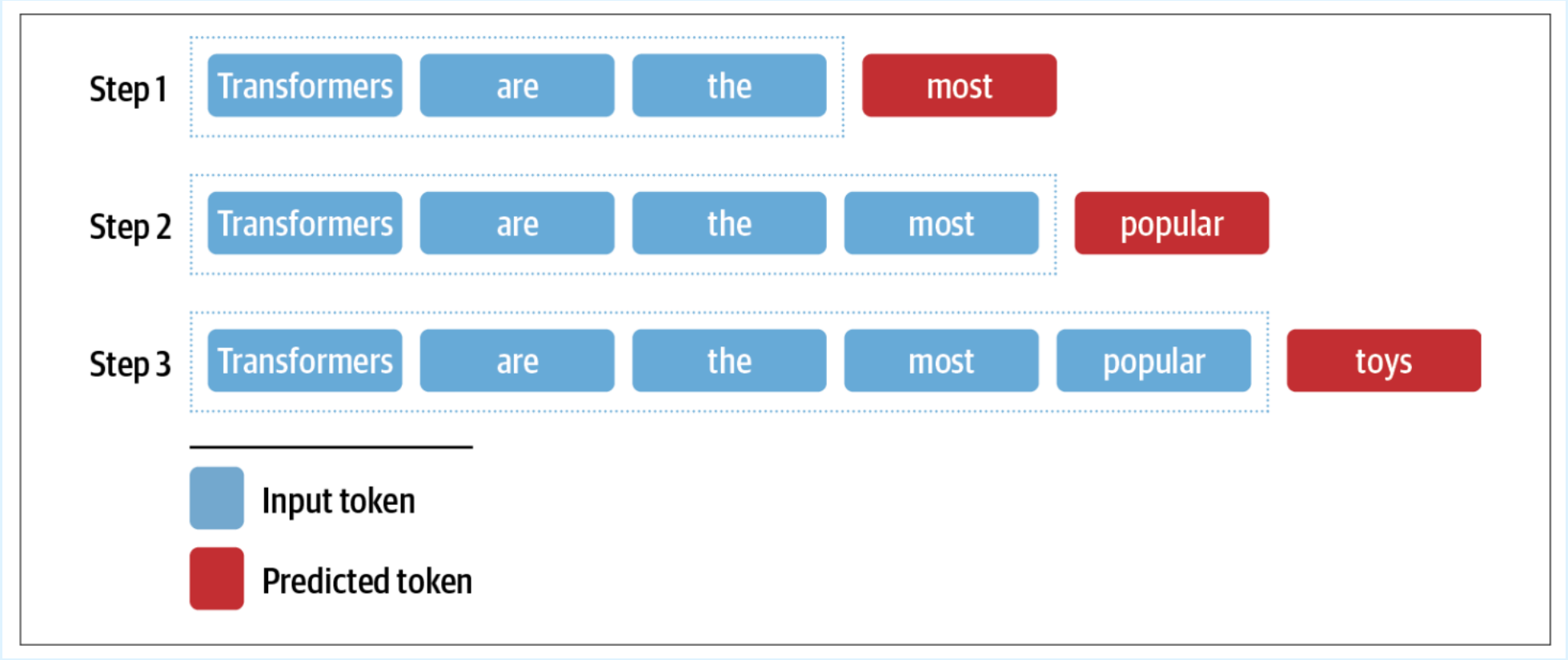


Session 7

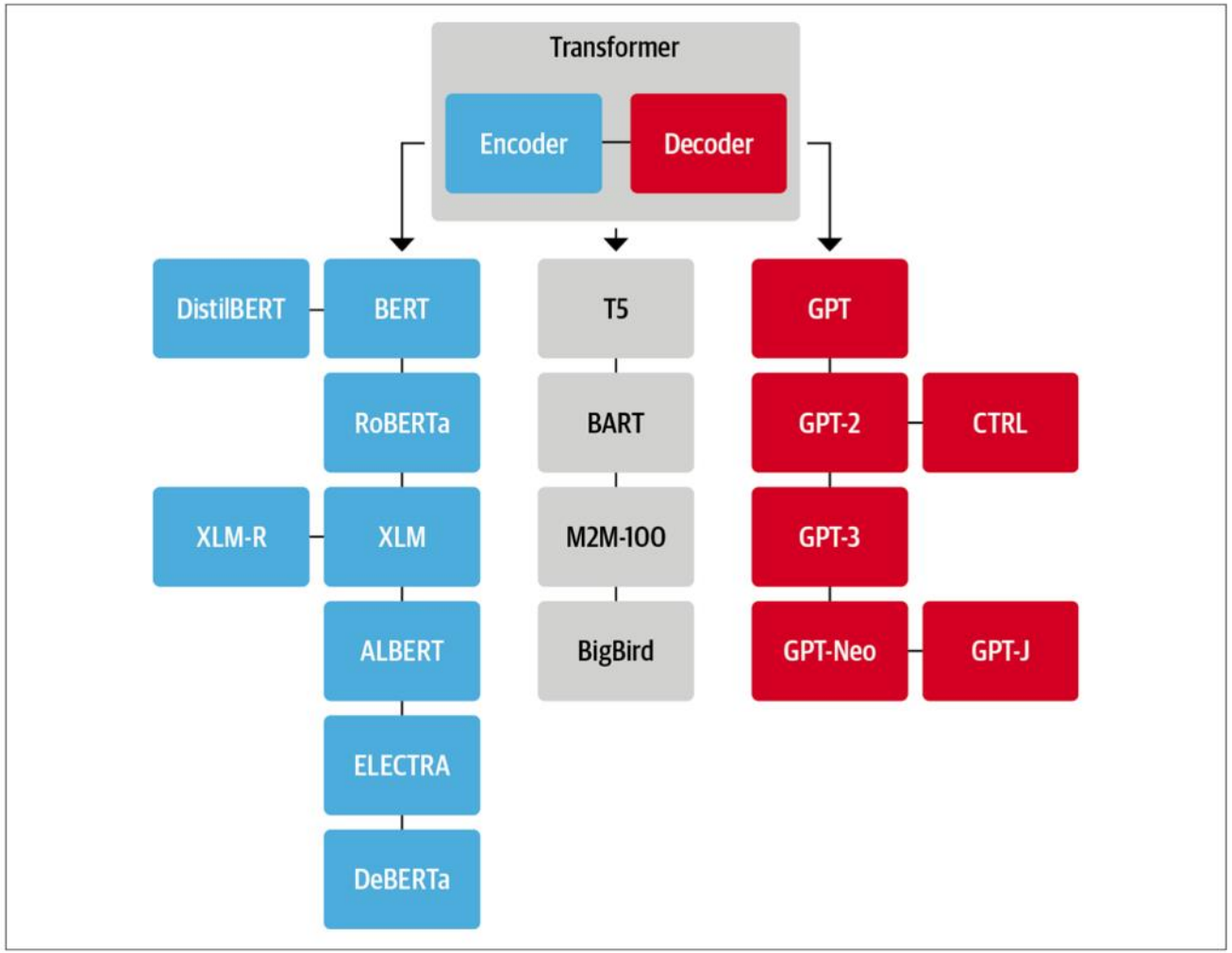
Retrieval Augmented Generation (RAG)

Introduction and Embeddings

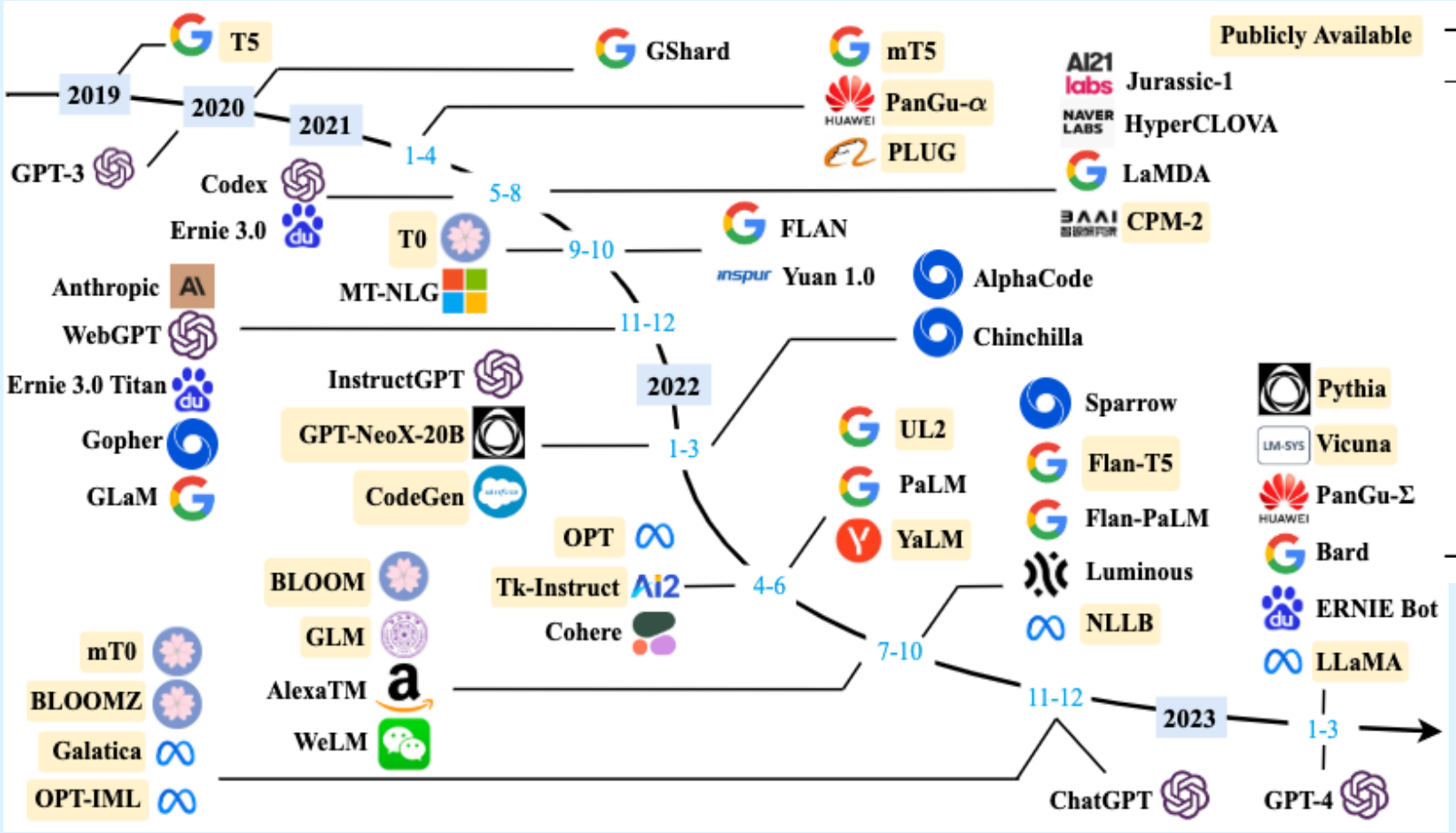
Refresher – The Decoder



Refresher: LLMs and the Transformer Model

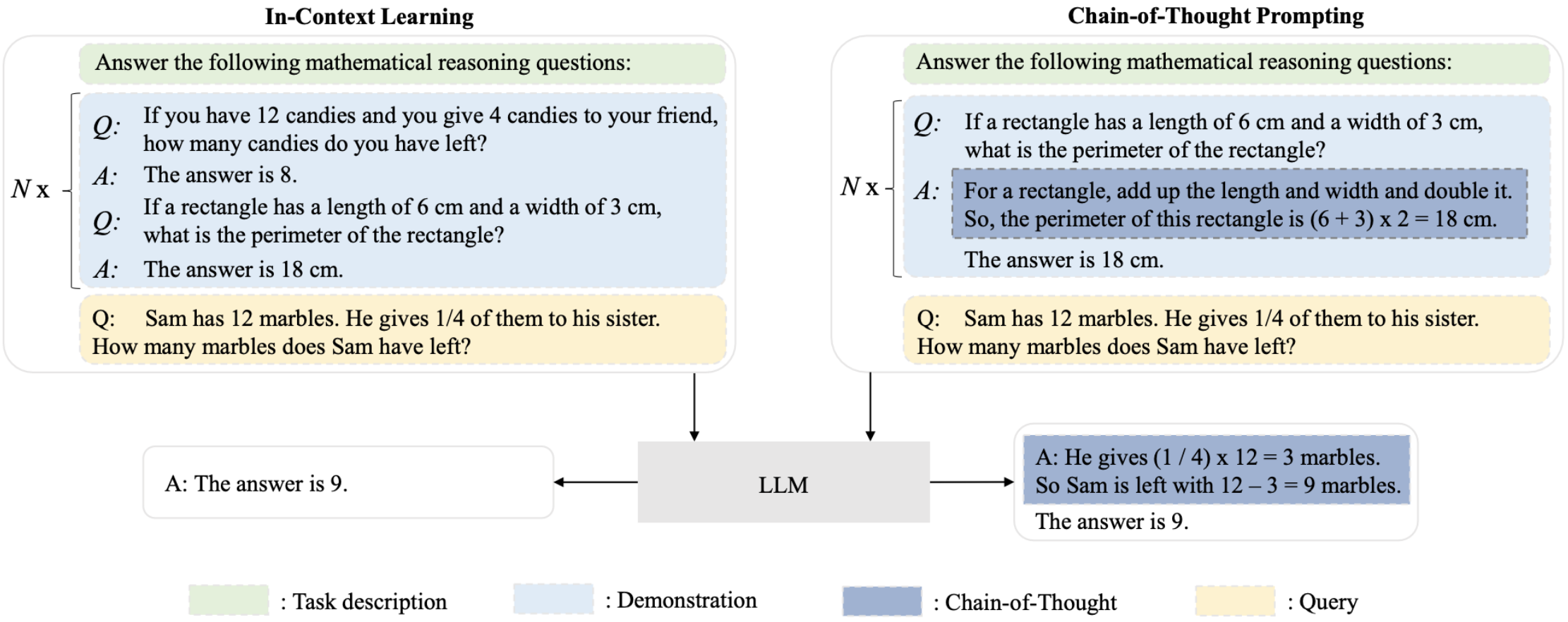


Refresher: Pre-Trained Language Model - Decoder



Model	Category	Size
GPT3 [55]	Casual decoder	175B
PanGU- α [73]	Casual decoder	207B
OPT [79]	Casual decoder	175B
PaLM [56]	Casual decoder	540B
BLOOM [66]	Casual decoder	176B
MT-NLG [90]	Casual decoder	530B
Gopher [59]	Casual decoder	280B
Chinchilla [34]	Casual decoder	70B
Galactica [35]	Casual decoder	120B
LaMDA [85]	Casual decoder	137B
Jurassic-1 [89]	Casual decoder	178B
LLaMA [57]	Casual decoder	65B
GLM-130B [80]	Prefix decoder	130B
T5 [71]	Encoder-decoder	11B

Refresher: Large Language Models



Conversational AI: What is it?

- Conversational AI helps machines interact with us naturally.
- Users can easily interact with conversational AI because this technology is designed to engage in human-like communication
- Chatbots, virtual assistants, and messaging apps like Alexa, Siri, Bixby, or Google Assistant—these can be categorized as conversational AI.



What is RAG?

ie UNIVERSITY

Retrieval

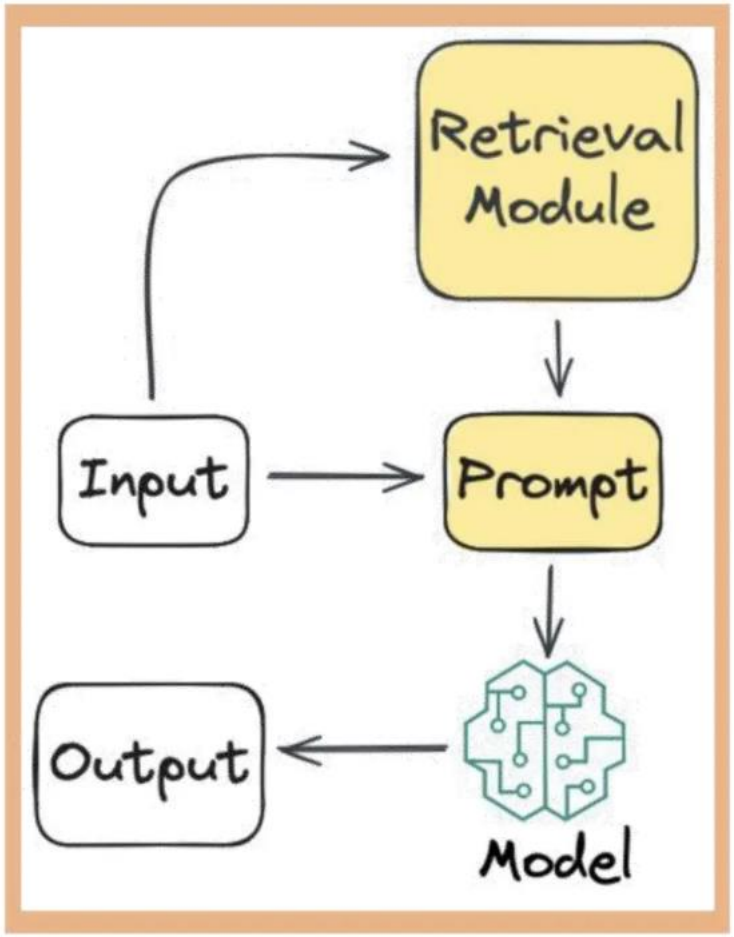
- Based on user query, fetches the relevant content from the external knowledge base

Augmentation

- Adding the retrieved context to the user prompt to use as an input to the foundation model

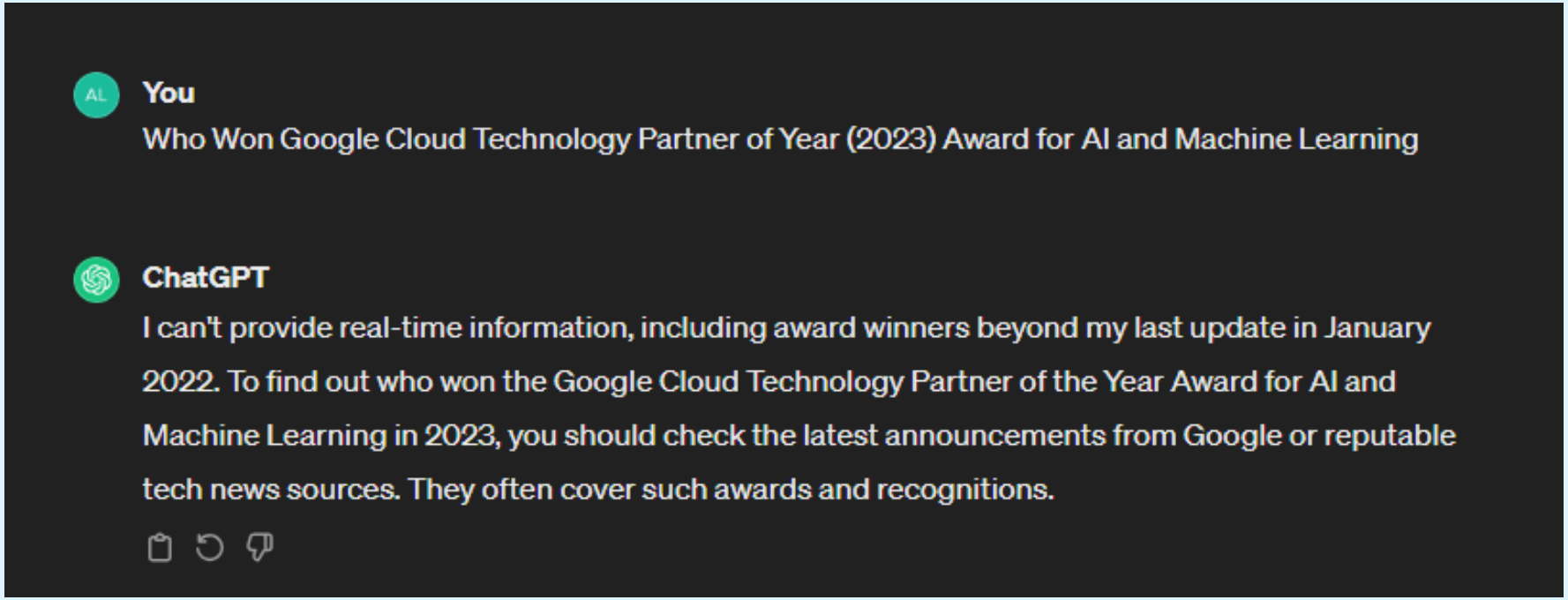
Generation

- Response from the foundation model based on the augmented prompt

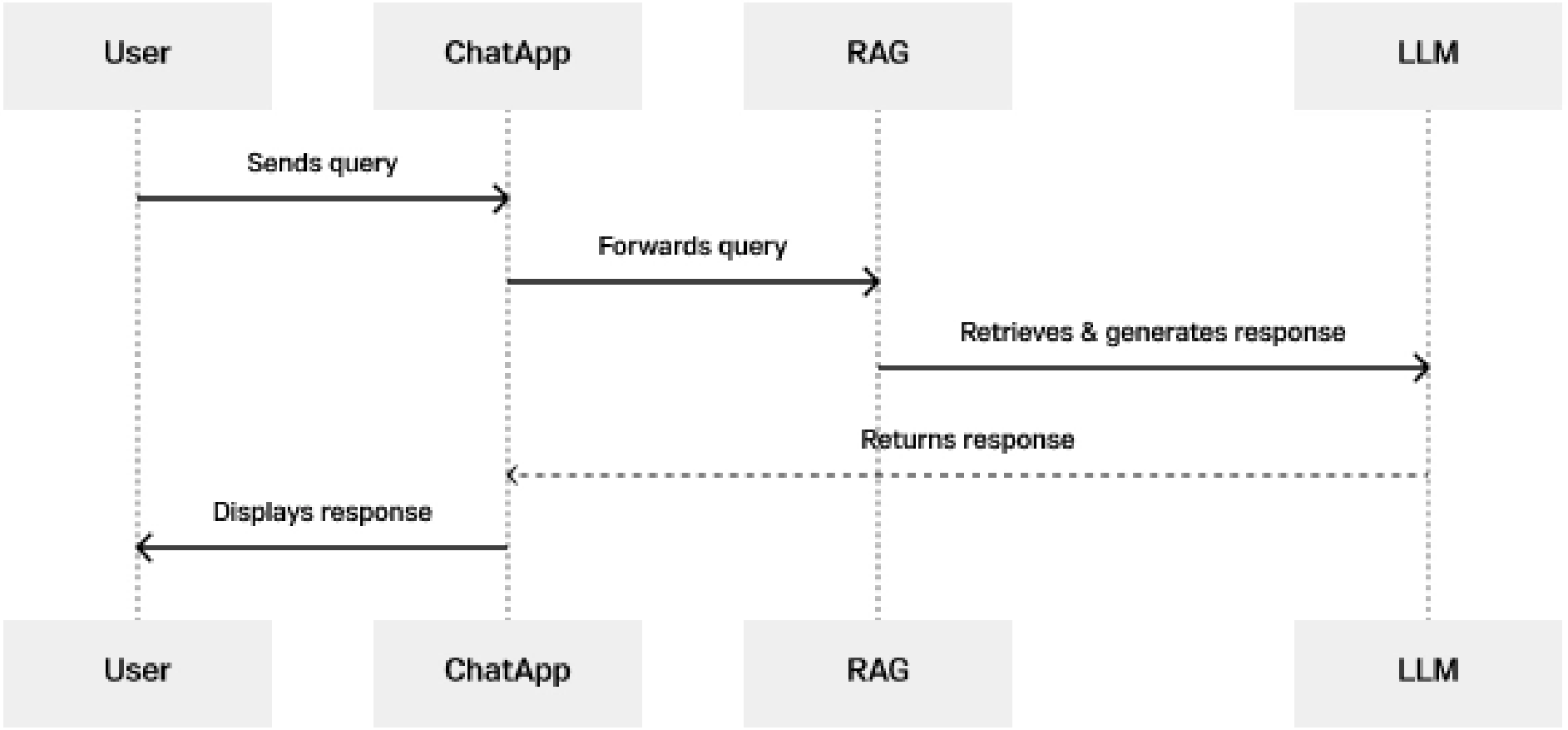


Why is RAG Important?

- LLM is like an over-enthusiastic new employee who refuses to stay informed with current events but will always answer every question with absolute confidence
- RAG solves some of the ”knowledge gap” by redirecting LLMs to retrieve relevant information from authoritative, pre-determined sources

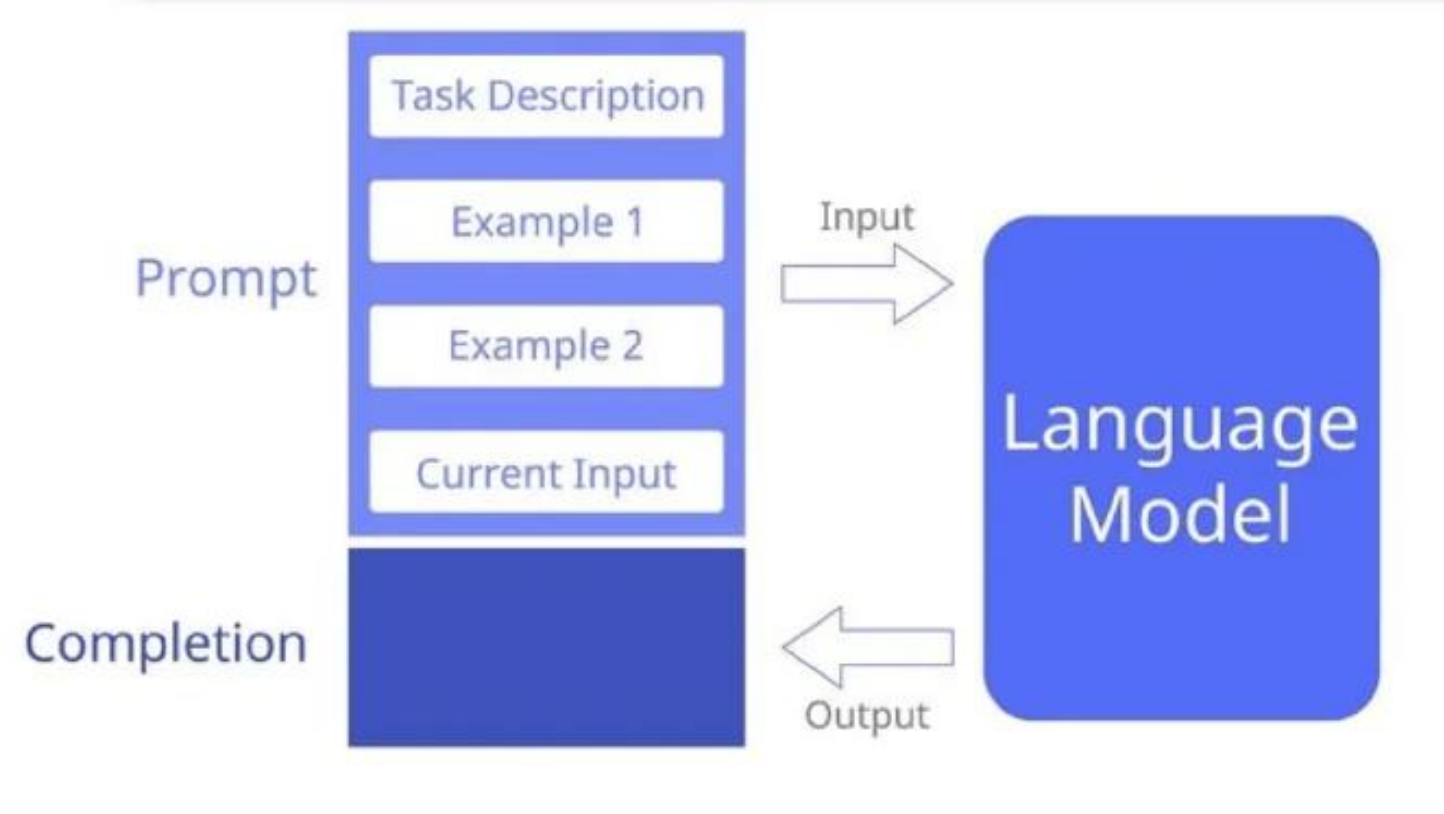


RAG-Based Chat Application



RAG and LLMs

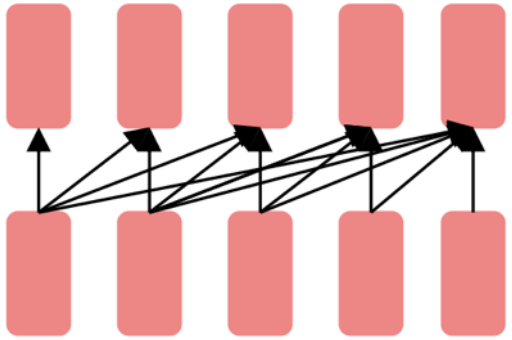
- An AI model that uses transformer architecture along with embeddings from vast datasets to understand, summarize, generate natural language



Capabilities

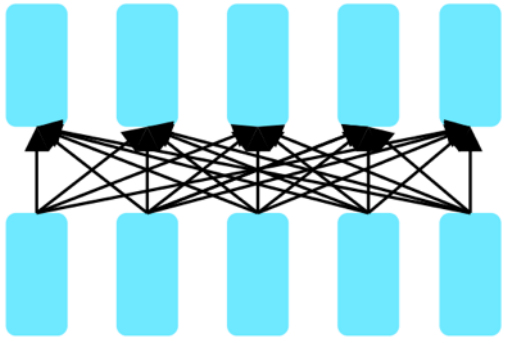
- Generation
- Summarization
- Translation
- Classification
- Question Answering

RAG and LLMs



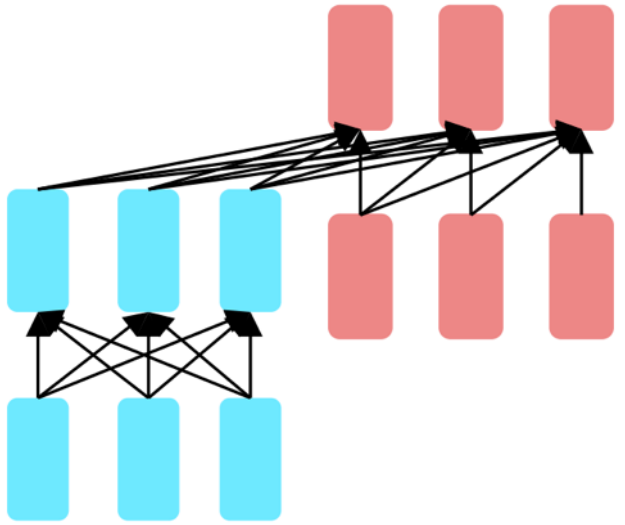
Decoders

- GPT
- Claude
- Llama
- Mistral



Encoders

- BERT
- HuBERT

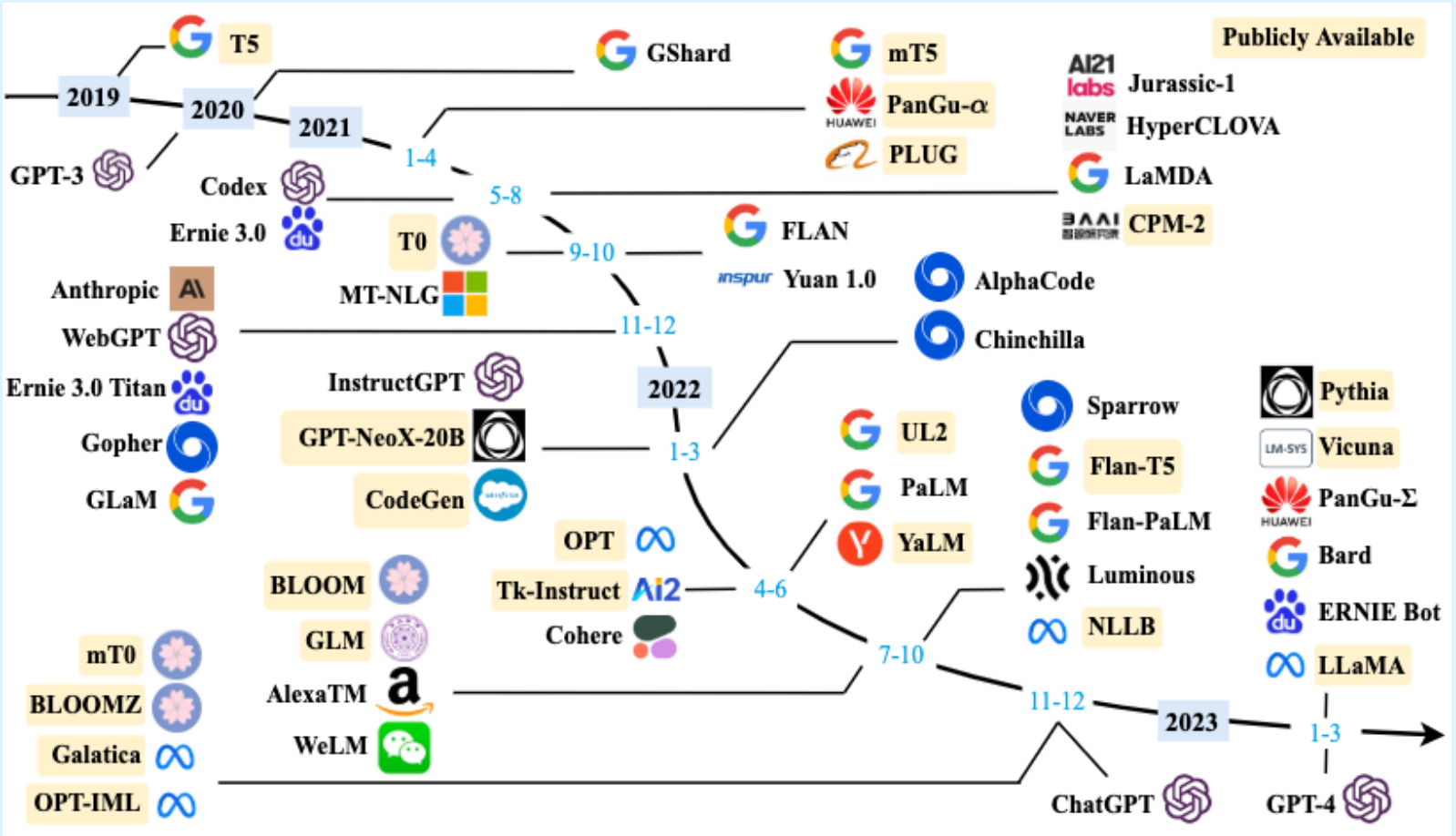


Encoder-Decoders

- Flan-T5
- Whisper

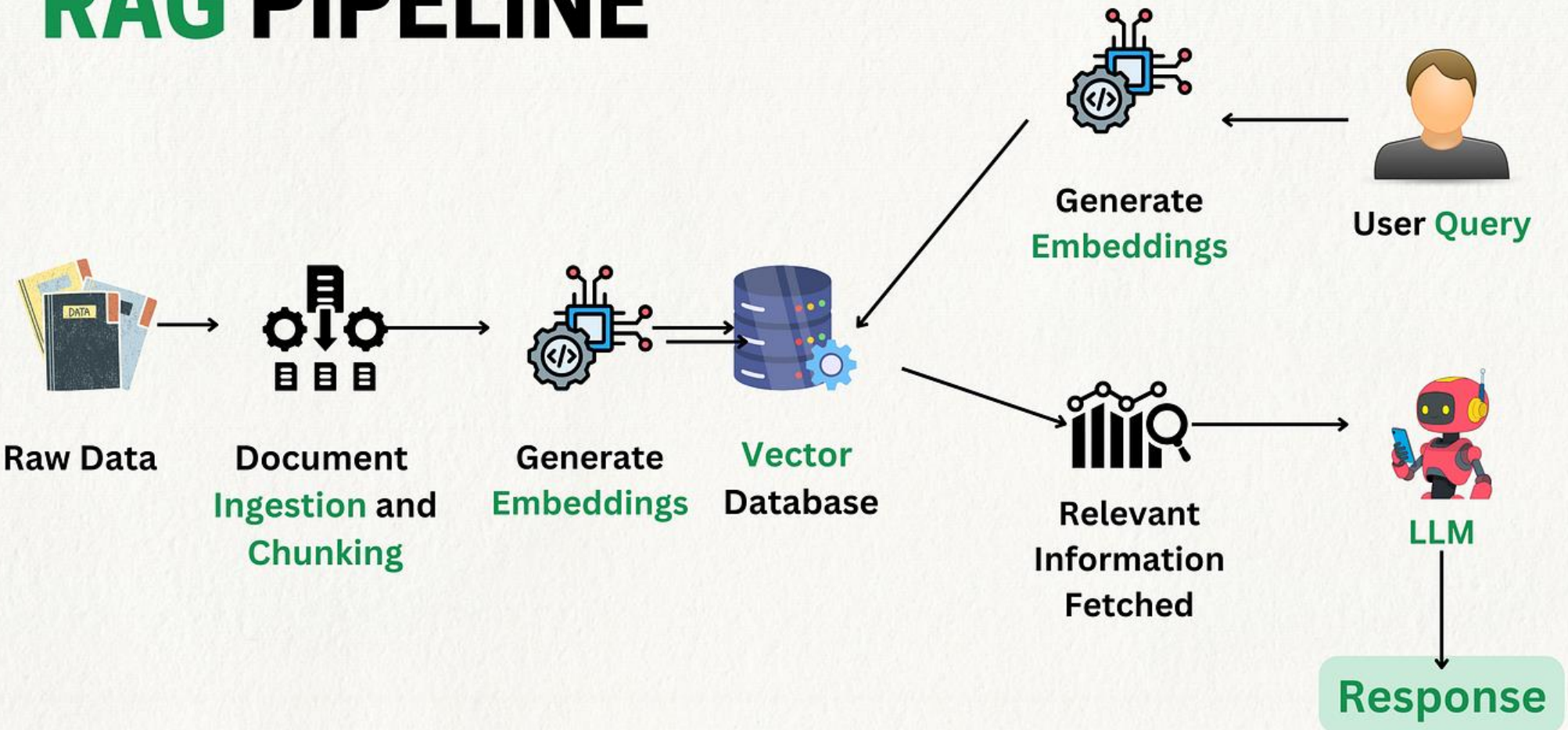
RAG Foundational Models

- Scaling PLMs (Pre-trained Language Models) by model size and/or data size often leads to an improved model capacity on downstream tasks

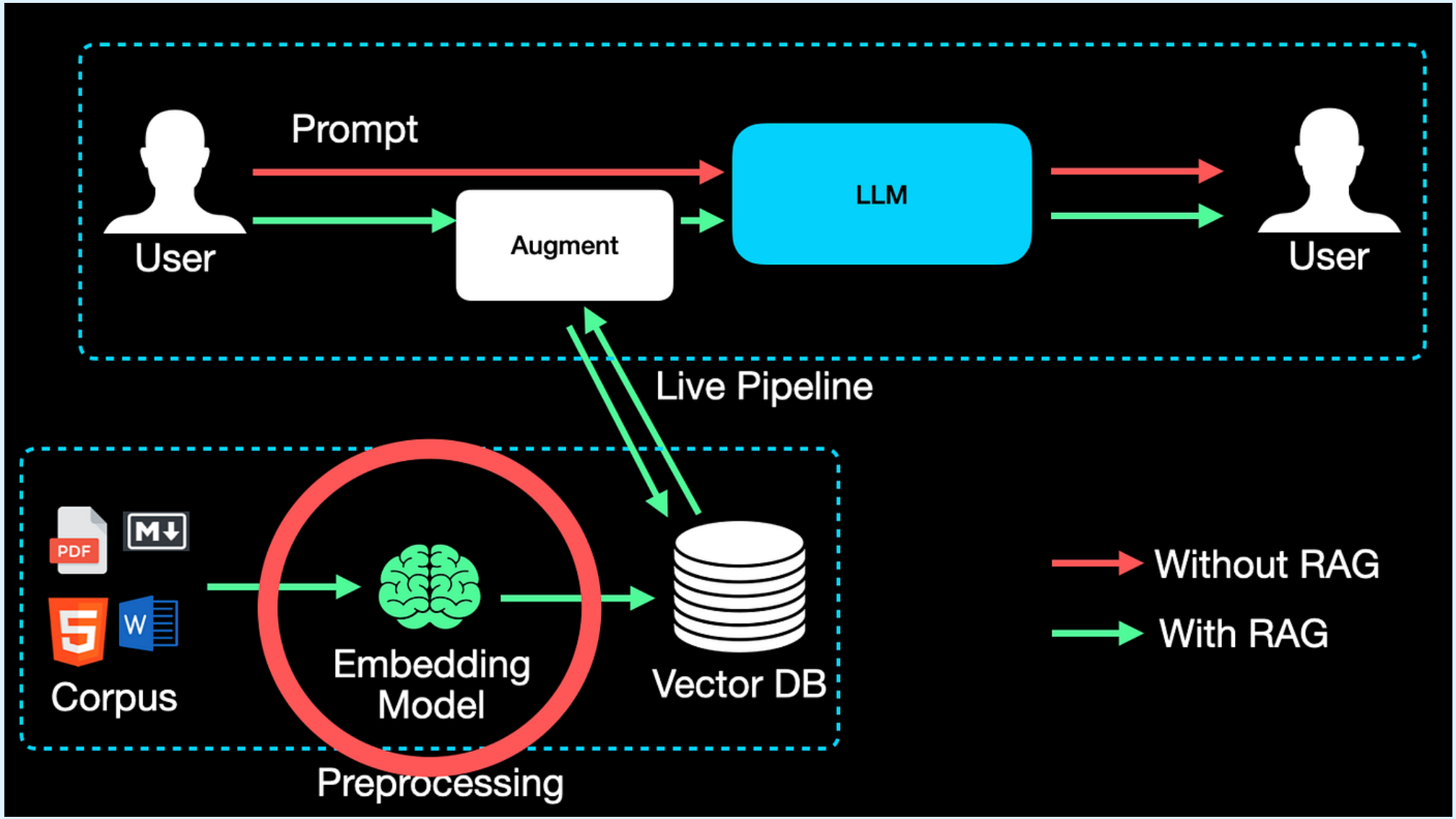


Model	Category	Size
GPT3 [55]	Casual decoder	175B
PanGU- α [73]	Casual decoder	207B
OPT [79]	Casual decoder	175B
PaLM [56]	Casual decoder	540B
BLOOM [66]	Casual decoder	176B
MT-NLG [90]	Casual decoder	530B
Gopher [59]	Casual decoder	280B
Chinchilla [34]	Casual decoder	70B
Galactica [35]	Casual decoder	120B
LaMDA [85]	Casual decoder	137B
Jurassic-1 [89]	Casual decoder	178B
LLaMA [57]	Casual decoder	65B
GLM-130B [80]	Prefix decoder	130B
T5 [71]	Encoder-decoder	11B

RAG PIPELINE

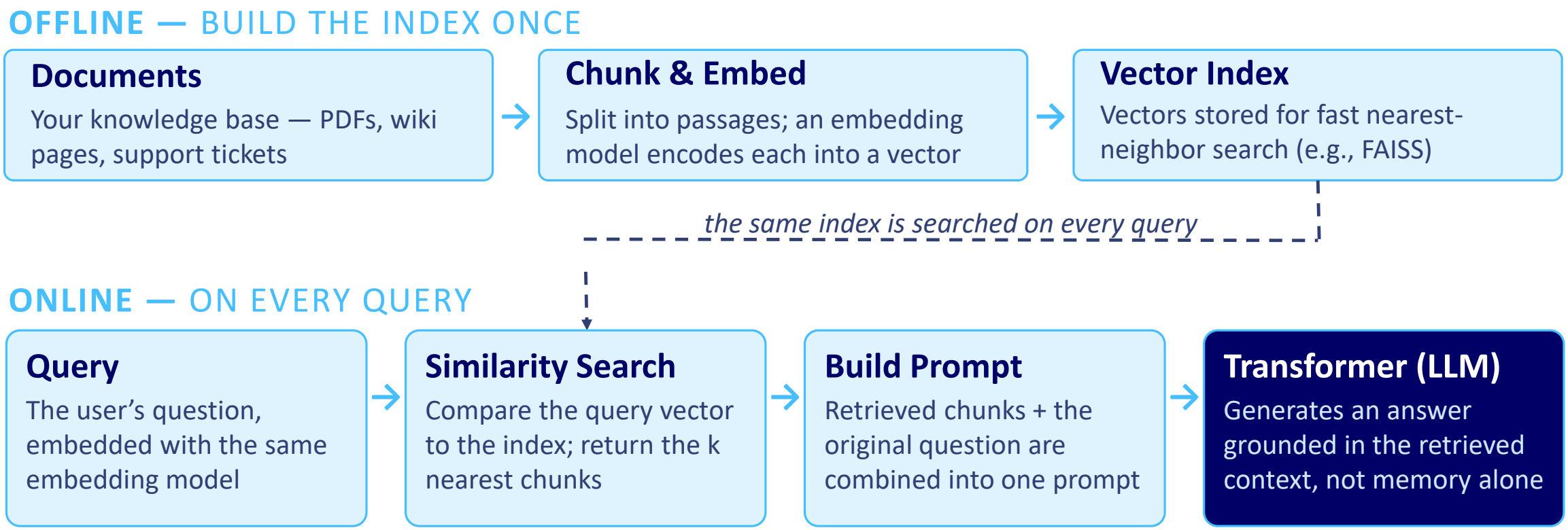


RAG Embeddings



RAG Architecture: Where Embeddings Fit

RAG wraps the same Transformer LLM with two extra stages: an offline step that indexes your documents, and an online step that retrieves from that index before generating.



Same model, new inputs: the LLM box above is the exact same Transformer decoder from earlier in this course — RAG only changes what goes into its prompt, not the model itself.

Sources: Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (2020); Johnson et al., "Billion-Scale Similarity Search with GPUs" (FAISS, 2017).

RAG Embeddings vs. Transformer Embeddings

Transformers already turn text into vectors internally. RAG reuses that same core idea — embeddings — for a very different job: finding relevant text, instead of predicting the next token.

Token Embeddings (Inside a Transformer)

- Each token is mapped to a vector capturing its meaning in context, then immediately by self-attention layers
- Contextual: the same word gets a different vector depending on the sentence around it
- Not meant to be compared across sentences — an internal representation, not a standalone output

RAG Embeddings (For Retrieval)

- A separate, usually smaller model encodes a whole chunk of text into one fixed-length vector for its overall meaning
- Built so semantically similar text lands at nearby vectors — enabling fast nearest-neighbor search
- The vector itself is the product: stored in an index and compared directly, e.g. with cosine similarity

Same idea, two jobs: dense vectors learned from data — one feeds attention inside a single forward pass, the other builds a searchable map across an entire document collection.

Sources: Vaswani et al., "Attention Is All You Need" (2017); Reimers & Gurevych, "Sentence-BERT" (2019); Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (2020).

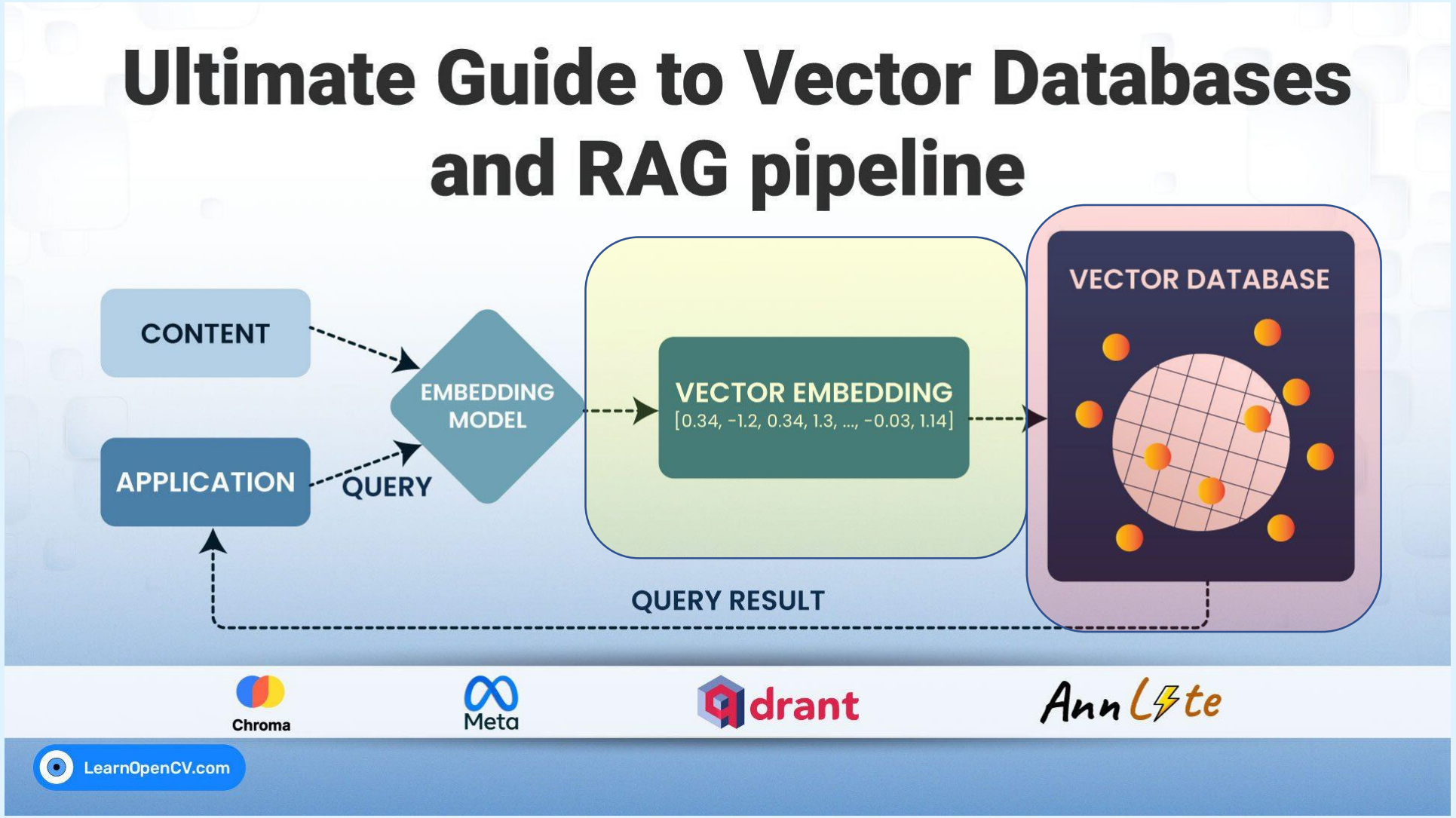


Session 7

Retrieval Augmented Generation (RAG)

Vector Databases and Semantic Search

Rag Vector Database



Vector Databases: Where RAG Stores Its Embeddings

The vector database is the piece from the architecture diagram that both the offline ingestion path and the online query path actually touch.

Document Chunks (Embedded)
Inserted into the DB once, offline

→

Vector Database
Finds the nearest vectors to a query — e.g., Chroma, Pinecone, Weaviate, Milvus

→

Top-k Matches
Returned chunks feed the prompt sent to the LLM

Querying a Vector Database in RAG (Chroma, Open Source)

```
import chromadb
from sentence_transformers import SentenceTransformer

client = chromadb.PersistentClient(path="./vector_db")
collection = client.get_or_create_collection("course_notes")
embedder = SentenceTransformer("all-MiniLM-L6-v2")
# (chunks were embedded and added to this collection in an earlier, offline step)

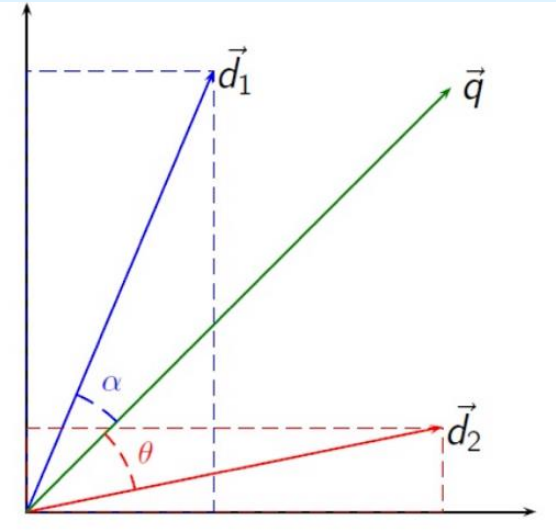
query = "How does LoRA reduce fine-tuning cost?"
results = collection.query(
    query_embeddings=embedder.encode([query]).tolist(),
    n_results=3,
)

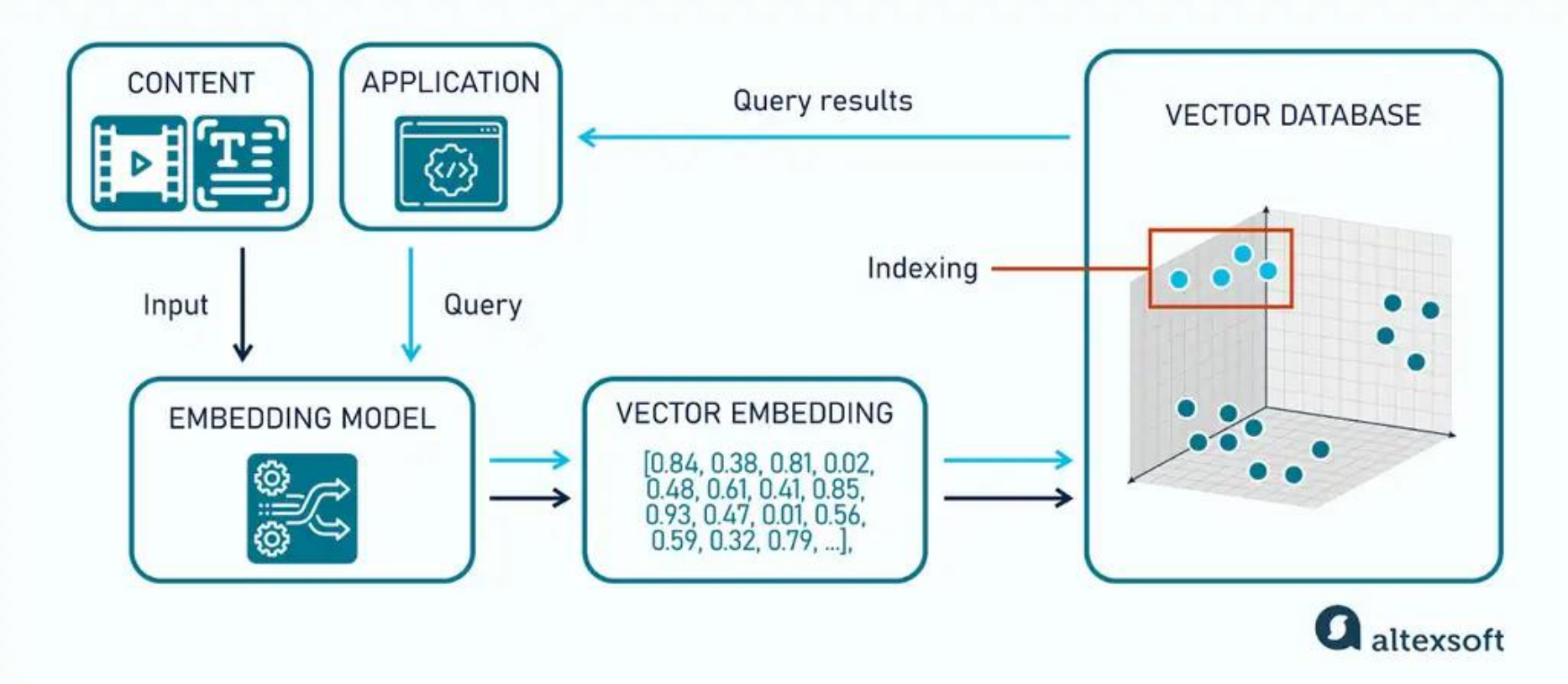
for doc, meta, dist in zip(results["documents"][0],
                           results["metadatas"][0], results["distances"][0]):
    print(f"[{dist:.3f}] ({meta['topic']}) {doc}")
```

A vector database = a vector index (for similarity search) + storage for the original text, metadata, and persistence across sessions — FAISS alone only provides the first part.

Sources: Johnson et al., "Billion-Scale Similarity Search with GPUs" (FAISS, 2017); Chroma, Pinecone, and Weaviate documentation.

RAG Vector Database and Semantic Search







Session 7

Retrieval Augmented Generation (RAG)

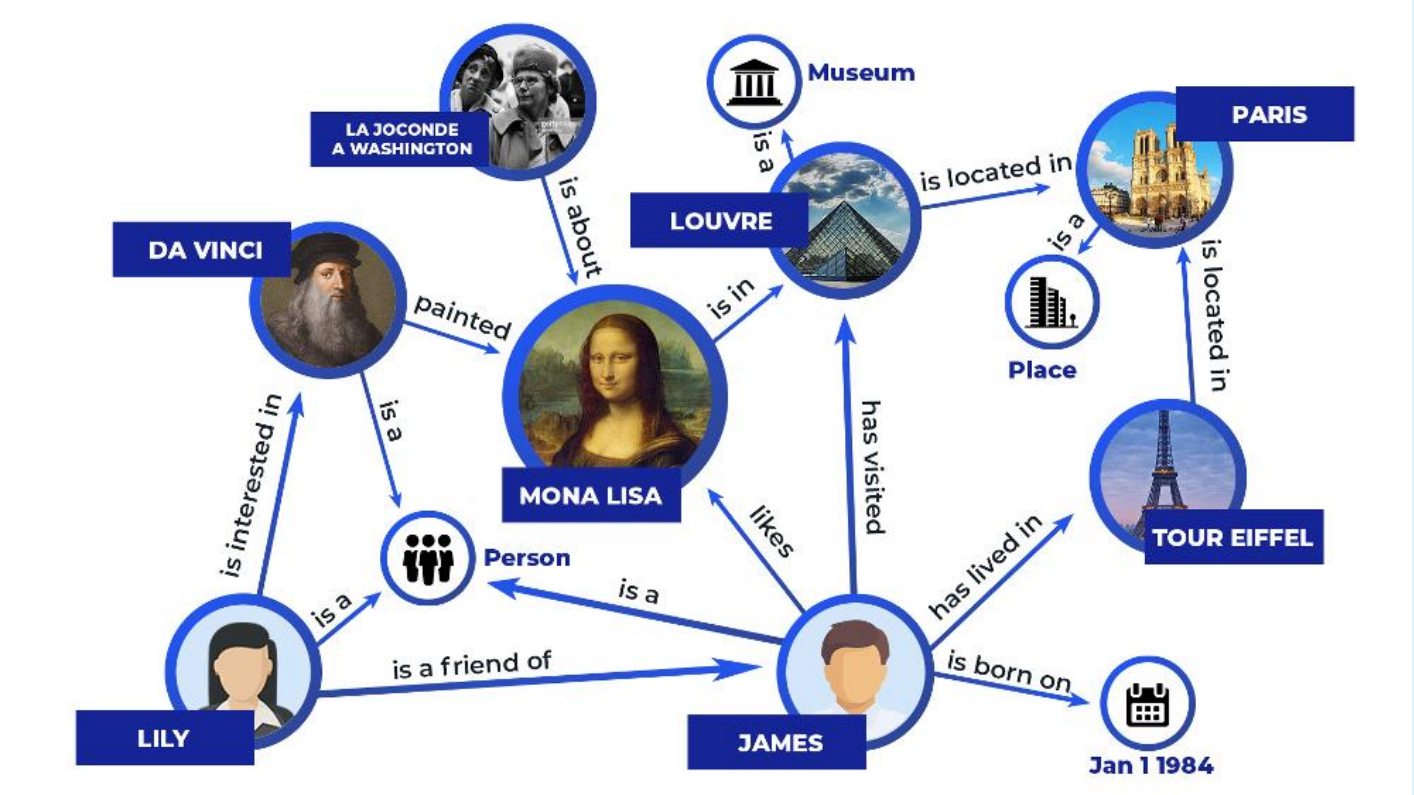
Knowledge Graphs

What are Knowledge Graphs?



“What artist is from Italy?”

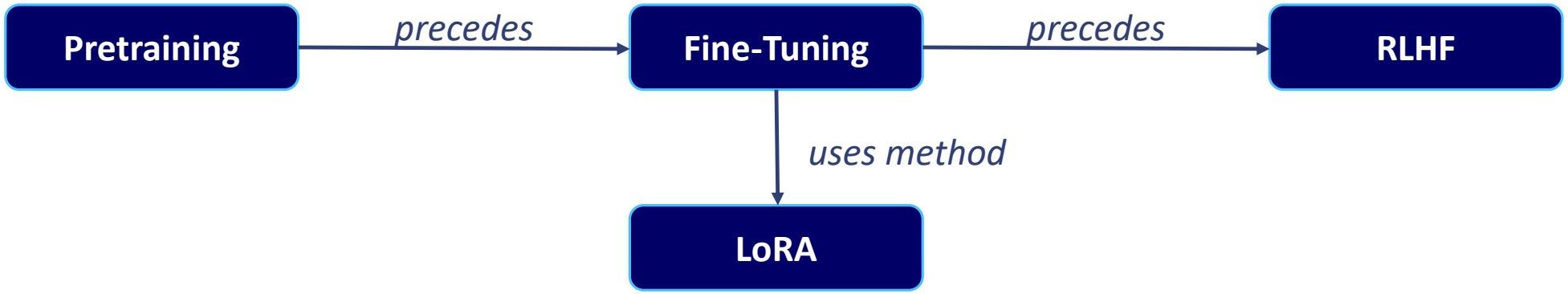
[0.22, -3.13, 5.88, 11.21]



- Knowledge graphs have encoded information that will allow us to query them.

Knowledge Graphs for RAG: How GraphRAG Works

Instead of (or alongside) chunks of text in a vector database, GraphRAG retrieves from a graph of entities and the labeled relationships between them.



- **Nodes and edges, not just chunks:** Entities (people, concepts, products) become nodes; the labeled relationships between them — "precedes," "uses method" — become edges
- **Retrieval by traversal:** A query is matched to relevant nodes, then the graph is walked along its edges to assemble connected facts — strong for multi-hop questions a single text chunk would miss
- **GraphRAG = graph + vectors:** Most production systems combine both: vector search for broad semantic recall, graph traversal for precise, structured, multi-hop reasoning — both are fed into the LLM's prompt

Sources: Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization" (Microsoft Research, 2024); Pan et al., "Unifying Large Language Models and Knowledge Graphs: A Roadmap" (2024).

Enterprise Knowledge Graph Solutions

Several vendors now package graph construction, storage, and GraphRAG-style querying as an enterprise platform.

Solution	Type	Focus
Neo4j (+ GraphRAG package)	Open-source core / Enterprise	Native graph database with the Cypher query language; the most common GraphRAG reference stack
Microsoft GraphRAG	Open-source (Microsoft Research)	LLM-built, community-summarized knowledge graphs; strong on broad "global" questions over large corpora
Amazon Neptune	Commercial (AWS)	Managed graph database (property graph + RDF); integrates with Bedrock for enterprise GraphRAG pipelines
Stardog	Commercial	Enterprise knowledge graph platform with built-in reasoning and a natural-language query layer
Ontotext GraphDB	Commercial	RDF triple store popular in regulated industries (pharma, publishing) for semantic search and RAG
data.world	Commercial	An "AI context platform" built on a knowledge graph layer over existing enterprise data catalogs

Sources: Neo4j, Microsoft Research GraphRAG, Amazon Neptune, Stardog, Ontotext GraphDB, and data.world product documentation.

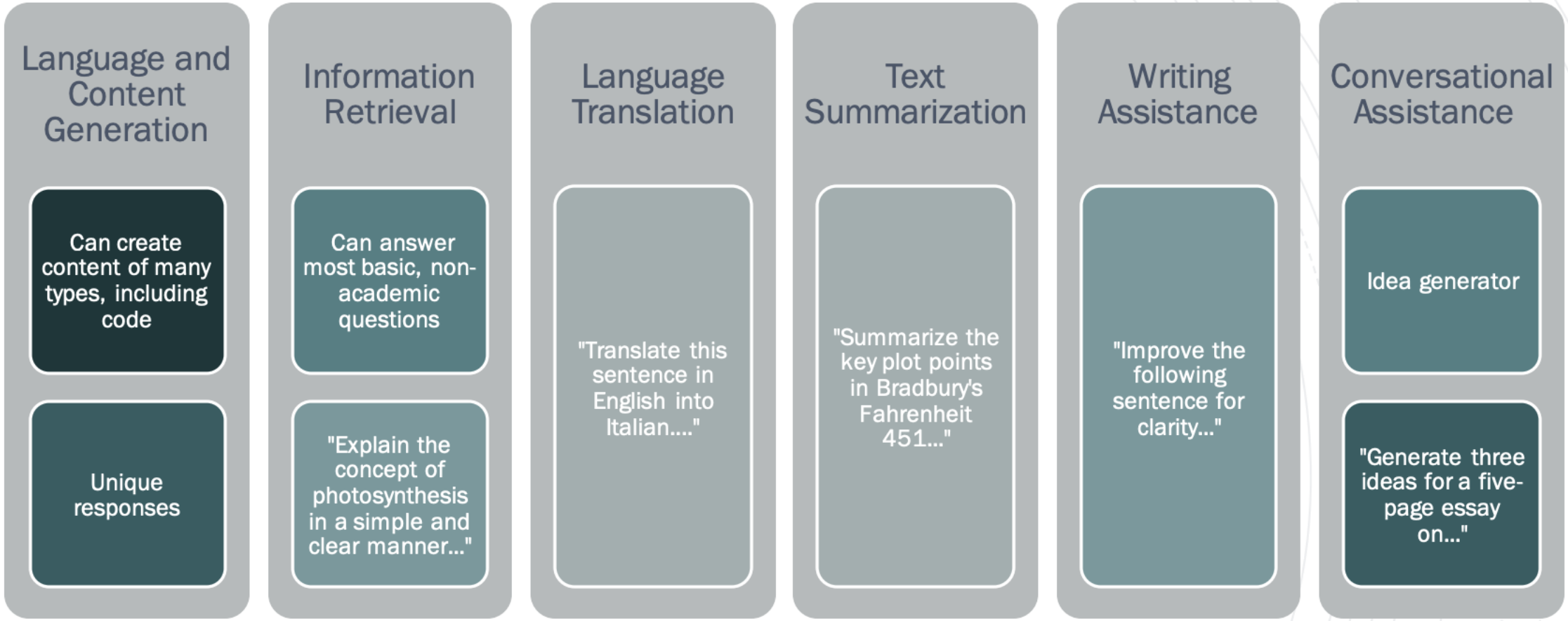


Session 7

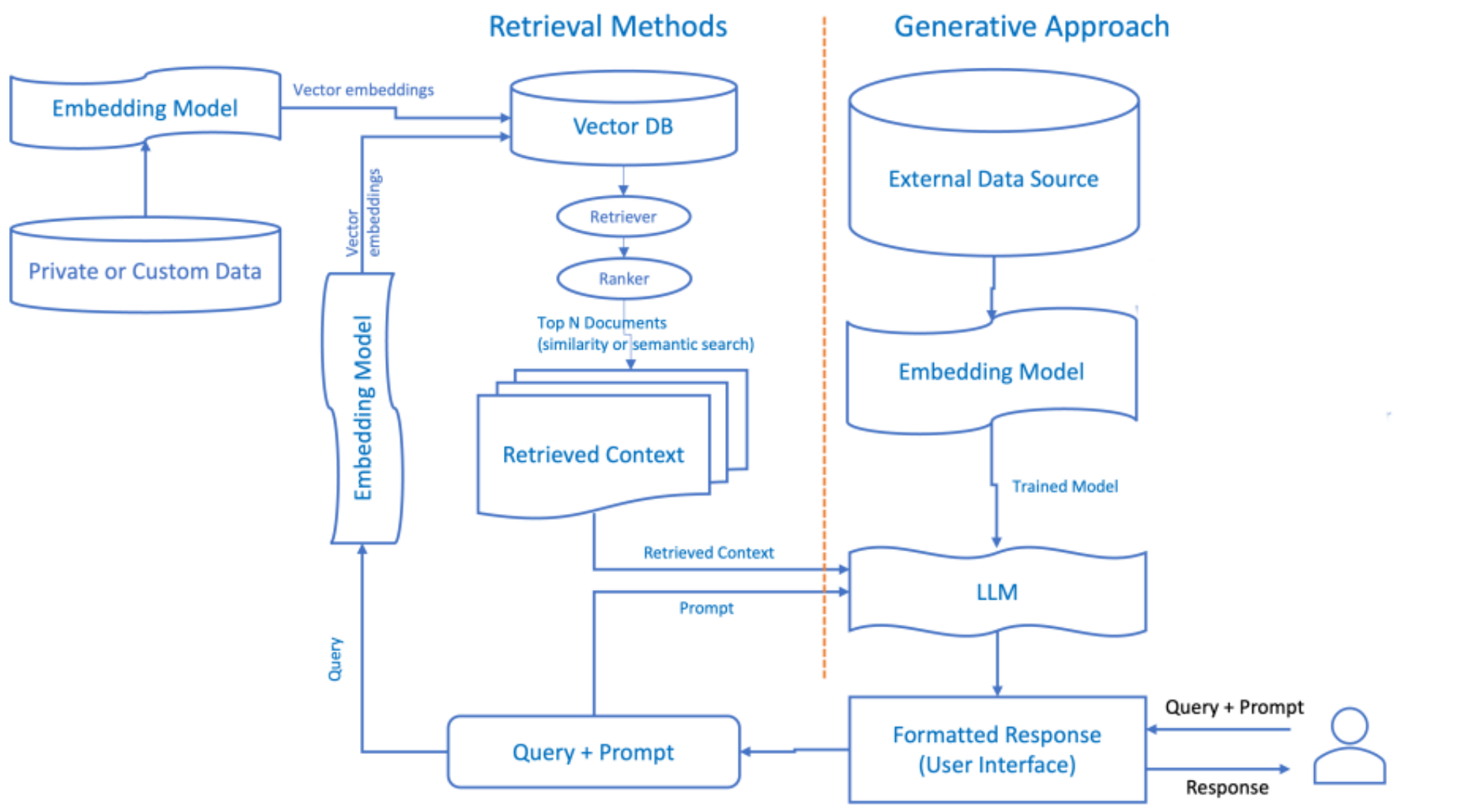
Retrieval Augmented Generation (RAG)

Retrieval Strategies

RAG: Popular LLM Functionality



General Information Retrieval Strategy



RAG Retrieval Strategies: The Basics

How you search the index matters as much as what's in it — most production RAG systems layer several of these strategies on top of each other.

- **Dense (semantic) retrieval:** Embed the query and compare it to stored vectors to find semantically similar text, even when the words don't match exactly — the approach covered so far in this course
- **Sparse (keyword) retrieval:** Classic term-matching methods like BM25 or TF-IDF, which excel at exact matches — names, codes, acronyms — that embeddings sometimes blur together
- **Hybrid retrieval:** Run dense and sparse search in parallel and fuse their results (e.g., Reciprocal Rank Fusion), combining semantic recall with keyword precision
- **Re-ranking:** Retrieve a larger initial set cheaply, then use a slower, more accurate cross-encoder model to re-score and reorder just those candidates before they reach the LLM

Sources: Robertson & Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond" (2009); Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering" (2020); Cormack et al., "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods" (2009).

RAG Retrieval Strategies: Going Further

Beyond a single similarity search, more advanced strategies reshape the query itself or chain several retrieval steps together.

- **Query transformation (HyDE):** Instead of embedding the raw question, first generate a hypothetical answer with the LLM, then embed and search with that — it often matches real passages better than the question's own wording
- **Multi-query retrieval:** Generate several reformulations of the same question, retrieve for each, then merge the results — widening recall before the top candidates are selected

- **Hierarchical / parent-child retrieval:** Search over small, precise chunks for accuracy, but return their larger surrounding section or document so the LLM gets fuller context

- **Iterative / multi-hop retrieval:** Retrieve, read, decide what's still missing, then retrieve again — letting an agent chain multiple retrieval steps for questions a single search can't answer

Sources: Gao et al., "Precise Zero-Shot Dense Retrieval without Relevance Labels" (HyDE, 2022); Trivedi et al., "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions" (IRCoT, 2022).

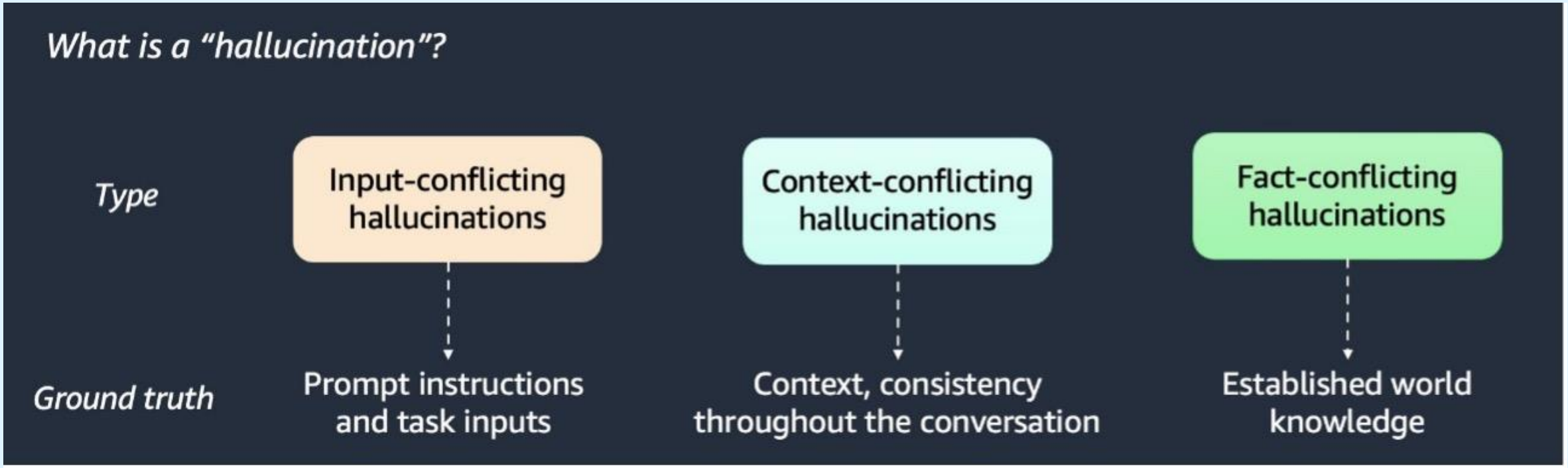


Session 7

Retrieval Augmented Generation (RAG)

RAG Evaluation and Hallucination

RAG Evaluation and Hallucination



*AWS Amazon Article: https://aws.amazon.com/blogs/machine-learning/detect-hallucinations-for-rag-based-systems

Detecting Hallucinations in RAG: Four Approaches

All four score each sentence of a generated answer against its retrieved context — a concrete, automatable version of the "faithfulness" idea from the evaluation slides.

- **LLM prompt-based detector:** An LLM reads the context and the answer, then scores each sentence from 0 (fully supported) to 1 (unsupported); a tuned threshold turns scores into hallucination/fact labels (e.g., Claude on Amazon Bedrock, via LangChain)
- **Semantic similarity detector:** Embeds the context and each answer sentence, then flags low cosine similarity as likely unsupported — the hallucination score is simply 1 minus the similarity (e.g., Amazon Titan Embeddings)
- **BERT stochastic checker:** Samples several answers to the same question from the same LLM, then compares sentences across samples with BERTScore — sentences that vary widely between samples are flagged as likely hallucinated
- **Token similarity detector:** Measures raw token overlap between answer and context with metrics like BLEU or ROUGE; low overlap flags a sentence as likely unsupported — the cheapest method, but the least semantically aware

*AWS Amazon Article: https://aws.amazon.com/blogs/machine-learning/detect-hallucinations-for-rag-based-systems



Session 7

Retrieval Augmented Generation (RAG)

Enterprise Search

Enterprise Search: RAG's Retrieval Layer at Company Scale

Enterprise search connects a RAG system to the data an organization already has — SharePoint, Confluence, Slack, email, ticketing systems, databases — instead of one hand-built knowledge base. The key addition beyond the vector DB and knowledge graph patterns already covered: it enforces each user's existing permissions, so retrieval only ever returns what that person is already allowed to see.

Company	Type	Focus
Glean	Commercial	Enterprise search platform purpose-built for AI, with 100+ permission-aware connectors
Microsoft 365 Copilot	Commercial (Microsoft)	Searches SharePoint, Teams, and Outlook via Microsoft Graph, enforcing existing M365 permissions
Google Vertex AI Search	Commercial (Google Cloud)	Managed enterprise search/RAG API connecting Google Workspace and third-party data sources
Amazon Kendra	Commercial (AWS)	Managed enterprise search service with natural-language queries and pre-built data-source connectors
Elastic (ESRE)	Open-source core / Commercial	Widely used open-source search engine, extended with hybrid and AI-powered retrieval features

Sources: Glean, Microsoft 365 Copilot, Google Vertex AI Search, Amazon Kendra, Elastic, and Coveo product documentation.



Session 7

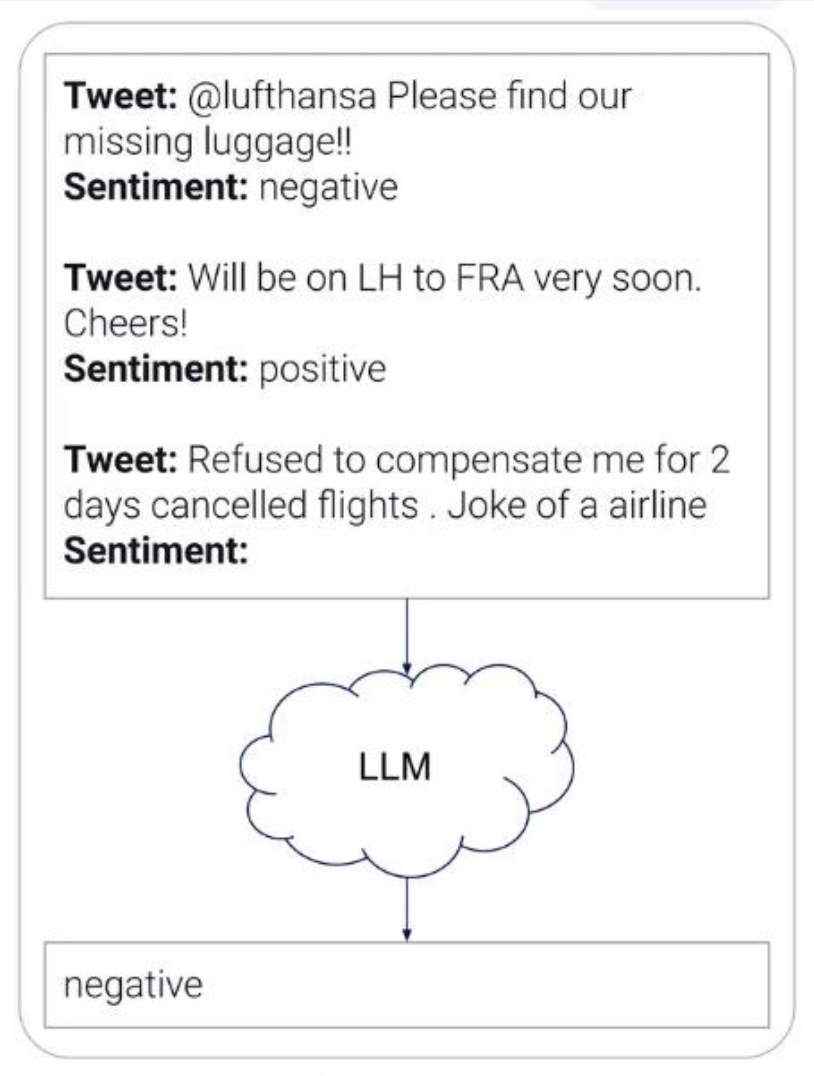
Retrieval Augmented Generation (RAG)

Building a RAG Pipeline

RAG Prompting Using LLMs

…aka **In-Context Learning (ICL)** involves providing a few input-output examples in the input context before asking the model to perform the task for an unseen example

- Zero-shot
- One-shot
- Few-shot
- Chain-of-Thought



RAG Types of Prompting

Zero-Shot Prompting

- Providing a direct instruction without any example context

Translate the following sentence into French: 'Hello, how are you?'

One-Shot Prompting

- Providing an example with the instruction to guide the output

Q: Translate 'Thank you' to French.

A: 'Bonjour'

Q: Translate 'How are you?' to French.

Few-shot Prompting

- Providing 2+ examples with the instruction to guide the output

Q: Translate 'Good morning' to French.

A: 'Bonjour'

Q: Translate 'Thank you' to French.

A: 'Merci'

Q: Translate 'How are you?' to French.

RAG Prompting Using LLMs

Chain-of-Thought (CoT) Prompting

- Encouraging the model to articulate intermediate reasoning steps before arriving at the final answer

Prompt:

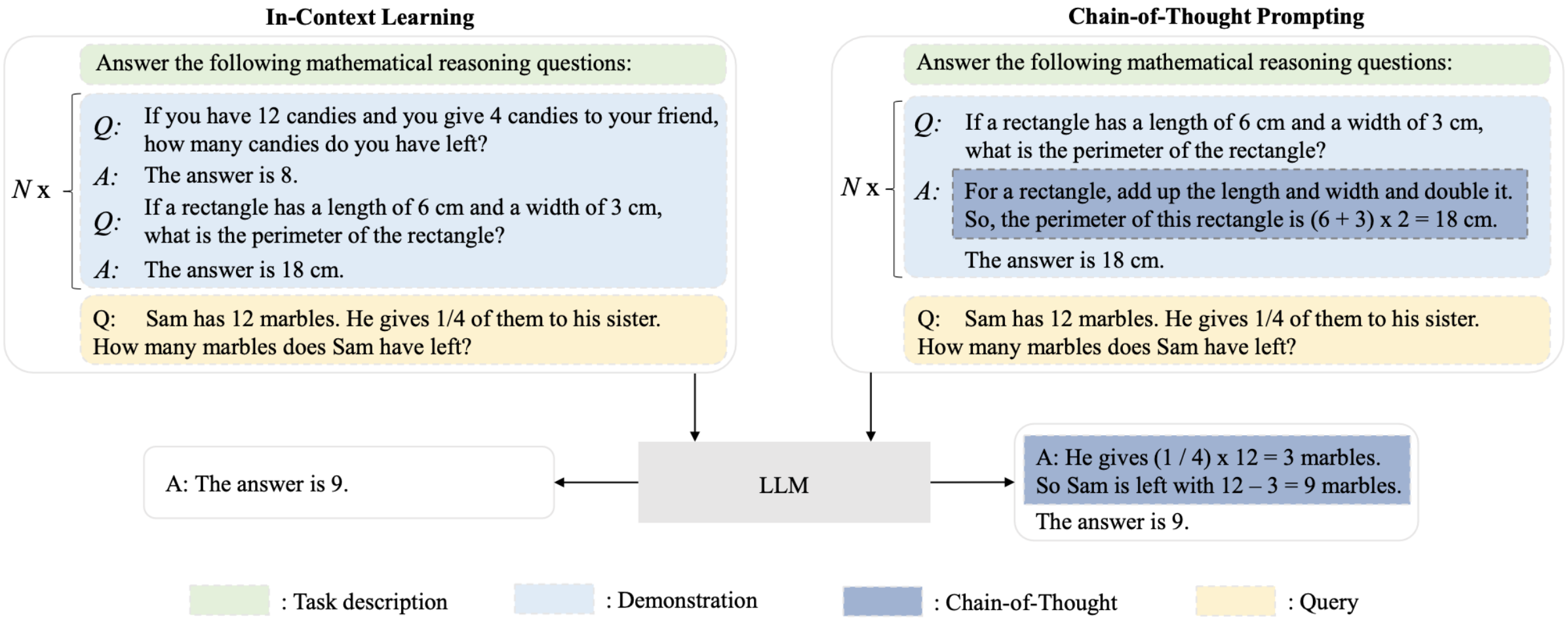
Solve the following problem and explain your reasoning:

If you have 5 apples, and you give away 2, how many apples are left?

Response:

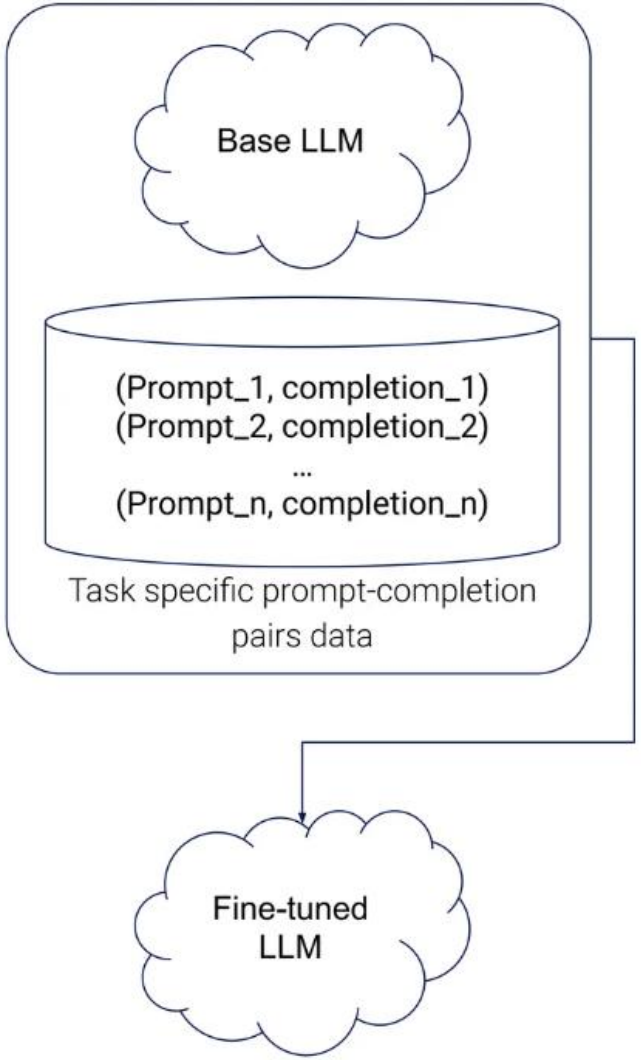
I start with 5 apples.
Then I subtract the 2 apples given away
5 - 2 equals 3 apples remaining

RAG: In-Context Learning (ICL) vs COT

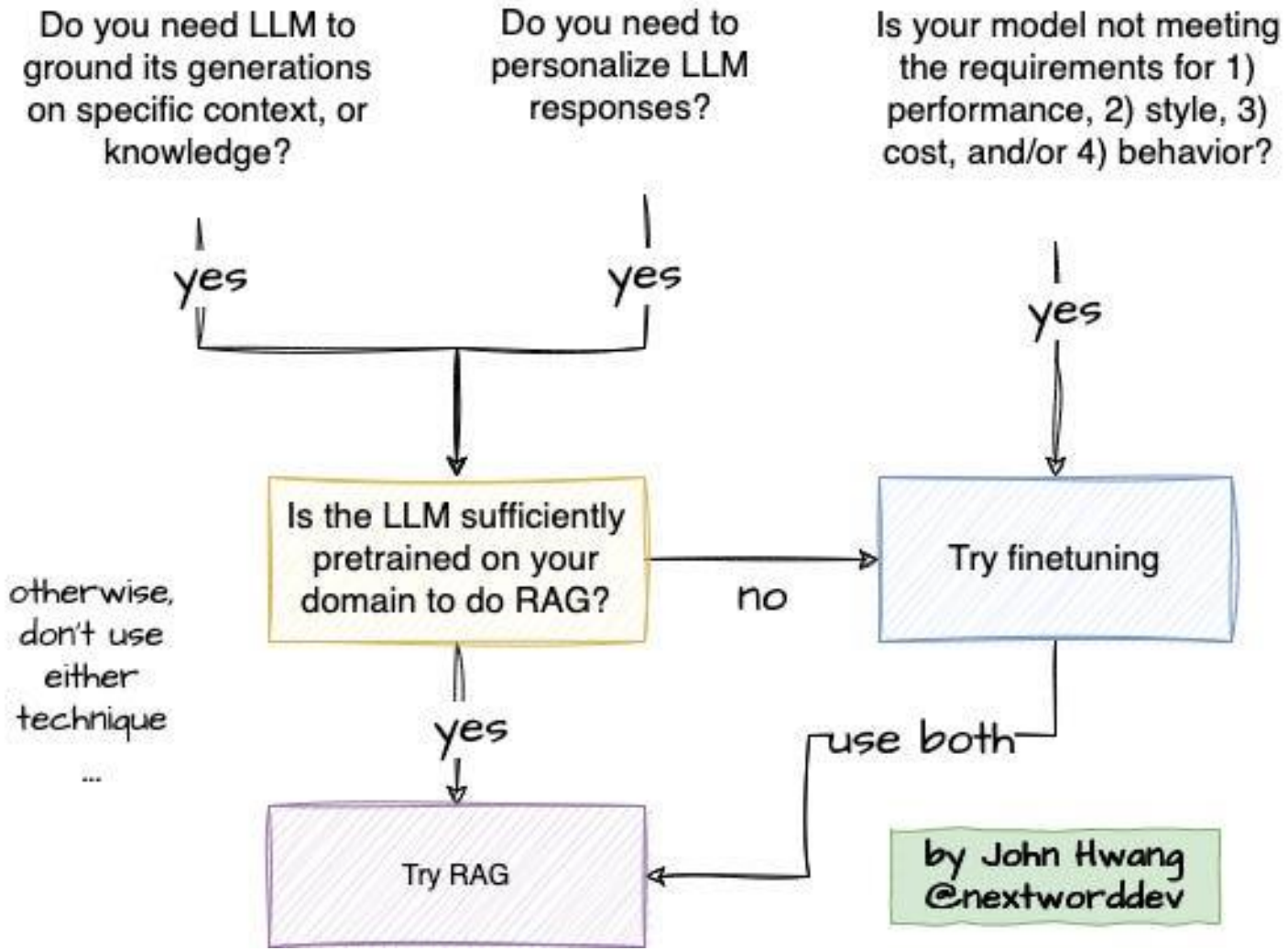


What it takes to fine-tune a RAG model

- A supervised learning process applied to foundational model to create a custom LLM
- Learns from task-specific (e.g. sentiment or summarization) prompt input-output pairs
- Extremely resource demanding (GPU power)
- Fine-tuning on a single task may lead to “catastrophic forgetting”
- Fine-tuning on multiple tasks requires a lot of training data



RAG Fine-Tune or Not?



Fine-Tuning the RAG Generator on a GPU

Fine-tuning the generator itself — not just picking a good base model — teaches it to stay grounded in retrieved context, cite sources, and say "I don't know" when the context doesn't cover the question. QLoRA (4-bit quantization + LoRA) makes this feasible on a single GPU.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from transformers import TrainingArguments, Trainer
from peft import LoraConfig, get_peft_model

bnb_config = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_compute_dtype=torch.bfloat16)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3-8b", quantization_config=bnb_config, device_map="cuda:0")
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3-8b")

lora_config = LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj","v_proj"])
model = get_peft_model(model, lora_config)   # only ~0.1–1% of weights become trainable

# Training examples shaped like a RAG prompt: context + question + grounded answer
def format_example(ex):
    prompt = f"Context:\n{ex['context']}\n\nQuestion: {ex['question']}\nAnswer:"
    return tokenizer(prompt + " " + ex["answer"], truncation=True, max_length=1024)

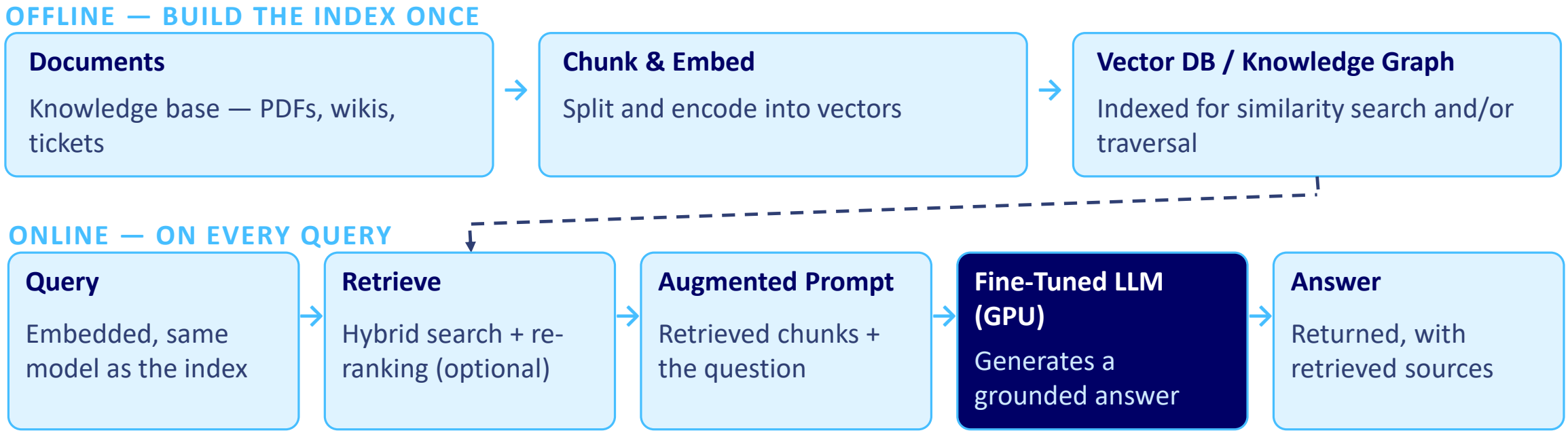
args = TrainingArguments(
    output_dir="./rag-llm-lora", per_device_train_batch_size=2,
    gradient_accumulation_steps=8, bf16=True, num_train_epochs=3, learning_rate=2e-4)

Trainer(model=model, args=args, train_dataset=raw_dataset.map(format_example)).train()
```

Sources: Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs" (2023); Hu et al., "LoRA" (2021); Hugging Face PEFT and TRL documentation.

The Final RAG Pipeline, End to End

Every piece covered in this section comes together into one system: an offline indexing path, and an online path that ends in the fine-tuned LLM generating a grounded answer.



Full circle: the LLM in the highlighted box is the exact model fine-tuned on the previous slide — everything upstream of it (embeddings, vector DB / knowledge graph, retrieval strategy) only changes what ends up in its prompt.

You will create a RAG pipeline in your next assignment!

This pipeline integrates the embeddings, vector database, knowledge graph, retrieval strategy, fine-tuning, and evaluation material covered across this section of the course.

Session 7

Quiz 1

20 minutes, please take spaces now

Thank you

Questions & discussion

John E. Ortega, Phd

johnevano@faculty.ie.edu

Office hours: By Appointment via Zoom

